Construction d'un arbre hiérarchique hybride

Génération de code frontend à partir de maquettes UX

Point d'avancement méthodologique

Sommaire

Contexte et objectif

Le problème de recherche

État de l'art étudié

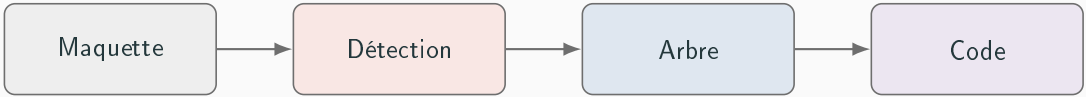
Notre proposition

Détection : une entrée image seule

Avancement et prochaines étapes

Contexte et objectif

Le pipeline général



Toute méthode passe par un modèle structuré intermédiaire : l'arbre hiérarchique.

La qualité du code final dépend presque entièrement de la qualité de cet arbre.

Objectif du mémoire

- Se concentrer sur l'étape la plus fragile du pipeline : la construction de l'arbre
- Proposer une méthode **hybride**, combinant précision géométrique et compréhension sémantique
- Livrables : un article de recherche **et** sa soutenance orale

Le problème de recherche

Deux familles, deux défauts opposés

Modèles multimodaux (MLLM)
Comprennent bien le **sens** des éléments

Hallucinent leur position exacte

Méthodes mathématiques
Précises sur la **position**

Aveugles au sens des éléments

Aucun travail existant ne sépare complètement et strictement ces deux rôles.

État de l'art étudié

Trois travaux de référence

Travail	Approche	Limite identifiée
DeclarUI	Vision par ordinateur + MLLM	Échoue sur les regroupements complexes (sections, listes) ; centré sur la navigation entre pages
ReverseORC	Découpage géométrique récursif (XY-Cut), depuis API d'accessibilité	Aucune étape sémantique ; ne part pas d'une image
DesignCoder	Arbre construit par l'IA en premier, géométrie calculée ensuite	Ordre inverse du nôtre ; dépend de métadonnées déjà connues

Le vide identifié

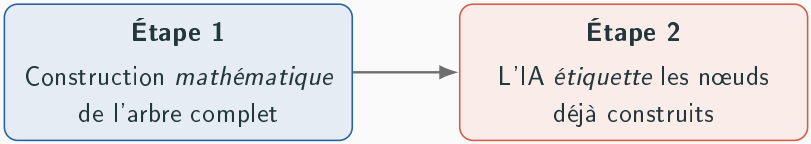
Aucune méthode ne garantit à la fois :

100% de précision
sur les positions

une vraie compréhension
des sections et listes complexes

Notre proposition

Une séparation stricte en deux temps



L'IA ne modifie jamais une position — elle attache seulement un nom.

C'est l'inverse exact de l'ordre choisi par DesignCoder.

Pourquoi cet ordre

- Le calcul garantit une structure **fiable et déterministe**
- L'IA n'a plus qu'un seul rôle simple : **nommer**, jamais positionner
- Les erreurs de raisonnement du MLLM ne peuvent plus contaminer la géométrie
- Approche **nouvelle**, pas une amélioration incrémentale d'un travail existant

Deux niveaux d'ambition pour l'étiquetage

Option A — priorité
Étiquetage sémantique simple de chaque nœud déjà construit

Option B — bonus
Détecter les sous-arbres similaires
→ composants réutilisables

L'Option A est développée et validée en premier pour garantir un résultat défendable dans les délais.

Détection : une entrée image seule

Contrainte choisie

- Le pipeline ne reçoit **que la capture d'écran** haute fidélité — aucune métadonnée de conception (Figma ou autre)
- Contrairement à ReverseORC (API d'accessibilité) et DesignCoder (métadonnées id/bbox/type/style), notre entrée reste une image pure
- Un éventuel code source réel ne sert que de **vérité terrain** pour l'évaluation — jamais donné au pipeline

Détecteur spécialisé plutôt qu'un MLLM

Détecteur spécialisé
Analyse des pixels, sans raisonnement

Position **fiable et déterministe**

MLLM pour la détection

Hallucine les positions, comme identifié dès le départ

Utiliser un MLLM ici casserait notre principe fondateur : la géométrie doit rester garantie par le calcul.

Preuve : le MLLM peut déterminer un type sémantique

- **DesignCoder** [2] : à partir d'un élément déjà localisé (bbox connue), un prompt dédié fait produire au MLLM une description sémantique précise — ex. un champ de recherche annoté comme suggérant *"search scope"*
- **ScreenCoder** [6] : son agent interroge le MLLM (*"Where is the sidebar?"*) qui renvoie une région **et** son étiquette

Nuance : ScreenCoder utilise le MLLM aussi pour la position — l'inverse de notre choix. Ses propres mécanismes de correction (NMS, heuristiques de secours) montrent que ce choix reste risqué sur la position.

Détecteurs utilisés dans la littérature

Travail	Détecteur	Ce qu'il fournit
DeclarUI [1]	Détection non supervisée + SAM	Régions, sans type direct
Vu et al. [5]	OCR puis SAM	Texte typé, régions visuelles non typées
Screen Parsing [4]	Faster R-CNN entraîné sur UI (AMP/RICO)	Position + type en une passe
ScreenCoder [6]	MLLM (grounding)	Position + type, mais non géométrique

Quel détecteur pour notre cas ?

Notre besoin précis
Un détecteur qui fournit **position + type grossier en une seule passe**, sans MLLM

- DeclarUI et Vu et al. reposent sur **SAM** : segmentation fiable, mais aucun type n'est produit — une étape supplémentaire serait nécessaire
- **Screen Parsing** [4] entraîne directement un détecteur d'objets (Faster R-CNN) à sortir des *"class-labeled bounding boxes"* — position et type en une seule passe, exactement notre besoin

Piste retenue : un détecteur entraîné de type Faster R-CNN, sur le modèle de Screen Parsing

Limite à garder en tête : entraîné sur des interfaces mobiles (AMP/RICO) — à vérifier sur des maquettes web

Bibliographie i

- Zhou, T. et al. *Bridging design and development with automated declarative UI code generation.* arXiv:2409.11667, 2024.
- Chen, Y. et al. *DesignCoder: Hierarchy-Aware and Self-Correcting UI Code Generation with LLMs.*
- Jiang, Y., Stuerzlinger, W., Lutteroth, C. *ReverseORC: Reverse Engineering of Resizable UI Layouts with OR-Constraints.* CHI 2021.
- Wu, J. et al. *Screen Parsing: Towards Reverse Engineering of UI Models from Screenshots.* UIST 2021.
- Vu, T.D., Hoang, C., Hy, T.S. *Multimodal Graph Representation Learning for Website Generation Based on Visual Sketch.* arXiv:2504.18729.

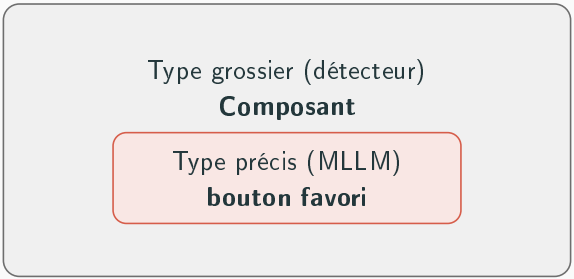
Bibliographie ii

Jiang, Y. et al. *ScreenCoder: Advancing Visual-to-Code Generation via Modular Multimodal Agents*. arXiv:2507.22827.

Ce que fournit le détecteur retenu

- Une **position exacte** (bbox) pour chaque élément
- Un **type grossier** : Texte / Image / Composant — **en une seule passe**, sans étape supplémentaire
- Rien de plus — aucune compréhension du sens

Le rôle du MLLM reste central



Pas de contradiction : le sens précis du MLLM est *contenu* dans le type grossier, jamais en désaccord avec lui.

Pourquoi pas de vérification croisée

- Une vérification détecteur $\leftrightarrow$ MLLM sur le type dirait presque toujours “d’accord” — complexité sans bénéfice réel
- Sans le MLLM, l’arbre resterait muet, comme Prototype2Code : bien rangé, mais incapable de dire ce que chaque partie représente
- Le MLLM reste **seul responsable** du sens — c’est notre contribution centrale, pas un rôle secondaire

Avancement et prochaines étapes

Où nous en sommes

1. **Cadrage du sujet** — terminé ✓
2. **Choix de la stratégie de détection** — terminé ✓
3. **Définition formelle de la structure de l'arbre** — en cours
4. Construction de l'algorithme géométrique
5. Intégration de l'étiquetage sémantique
6. Préparation des données de test
7. Mesure des résultats
8. Rédaction de l'article
9. Relecture et finalisation

Prochaine étape

Définir précisément ce que doit contenir
un nœud de notre arbre

en s'appuyant sur les structures déjà étudiées
(ReverseORC, DesignCoder)

Merci — questions et retours bienvenus